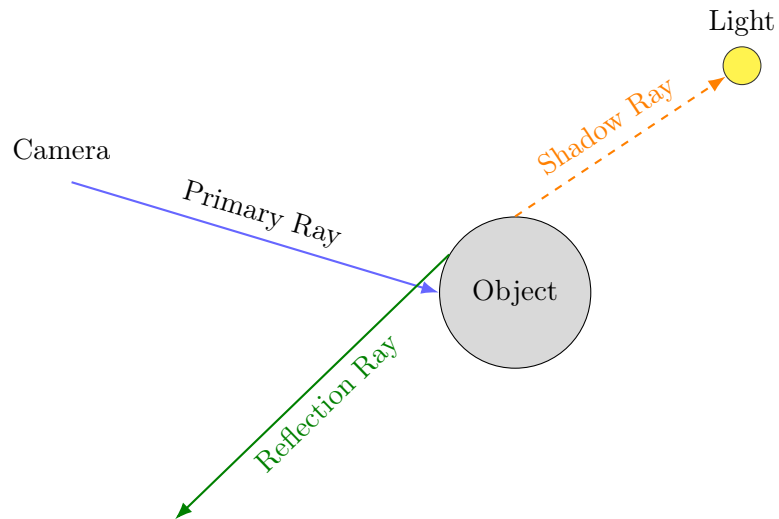


COMP2271: Computer Graphics

Part 6: Advanced Rendering Techniques

Lecturer: Frederick Li
Department of Computer Science
Durham University



How to Learn This Lecture

This module explores the cutting edge of rendering. To master these advanced concepts, focus on these mental models:

- **The Reverse Logic (Ray Tracing):** Flip your mental model of rendering. Instead of "projecting geometry to the screen" (Rasterisation), think about "shooting vision rays from the eye" (Ray Tracing). It is expensive but physically accurate.
- **The Texture as Data (FBOs):** A texture is not just a picture; it is a generic data array. Framebuffer Objects (FBOs) let us store anything, such as positions, normals, depth, into a texture. Shadow mapping works by rendering "distance from light" into a texture.
- **The Multi-Pass Mindset:** Advanced effects are rarely done in one go. Think in stages: "Pass 1: Capture the scene to a texture. Pass 2: Blur that texture. Pass 3: Combine it with the original."

1 Section 1: Ray Tracing Basics

1.1 A Different Philosophy: Rasterisation vs. Ray Tracing

Throughout this course, our primary focus has been on **rasterisation**, the dominant algorithm for real-time graphics used in applications like video games. Rasterisation is an *object-order* rendering technique. It takes 3D models, composed of triangles, and projects their vertices onto the 2D screen. The GPU then fills in the pixels covered by these triangles. Imagine a painter throwing buckets of paint (triangles) at a canvas (the screen). The final image is determined by which bucket of paint lands on top. It is incredibly efficient because its complexity is proportional to the number of triangles in the scene.

Ray Tracing, in contrast, is an *image-order* rendering technique that simulates the physical path of light in a more direct, intuitive way. Instead of projecting objects onto the screen, it works backwards from the camera. For every single pixel in the final image, we cast a virtual ray from the camera, through that pixel, and out into the 3D scene. We then determine what that ray hits. The colour of the pixel is determined by the properties of the object at the intersection point.

To use an analogy, if rasterisation is about throwing paint at a canvas, ray tracing is about looking at each point on the canvas and figuring out what colour of paint would have had to travel in a straight line to land there. This fundamental difference allows ray tracing to simulate complex optical phenomena like true reflections, refractions, and accurate shadows with a level of realism that is very difficult (though not impossible) to fake with rasterisation. The trade-off is performance; a basic ray tracer's complexity is proportional to the number of pixels multiplied

by the number of objects, making it historically too slow for real-time applications. However, modern GPUs now include dedicated hardware to accelerate these calculations, bringing real-time ray tracing into the mainstream.

1.2 The Core Components of a Ray Tracer

1.2.1 Defining a Ray

A ray is the fundamental primitive in a ray tracer. It is a semi-infinite line defined by a starting point (the **origin**) and a **direction** vector. We can describe any point along the ray with the following parametric equation:

$$P(t) = O + t\vec{D}$$

Where:

- $P(t)$ is a point on the ray.
- O is the origin vector of the ray.
- $\vec{D}$ is the normalised direction vector of the ray.
- t is a scalar parameter, where $t \geq 0$.

This can be represented in code with a simple class or structure.

Ray Class Definition

```
1 // Pseudo-code for a Ray class
2 class Ray {
3     constructor(origin, direction) {
4         this.origin = origin;
5         // A 3D vector (e.g., [x, y, z])
6         this.direction = normalize(direction);
7         // A normalized 3D vector
8     }
9
10    pointAt(t) {
11        // P(t) = Origin + t * Direction
12        return add(this.origin, scale(this.direction, t));
13    }
14 }
```

1.2.2 Ray-Sphere Intersection

The most critical operation in a ray tracer is calculating the intersection between a ray and the objects in the scene. The sphere is the simplest non-trivial shape for this calculation. A sphere is defined by its centre C and its radius r . Any point P on the sphere satisfies the equation:

$$(P - C) \cdot (P - C) = r^2$$

To find the intersection, we substitute the ray equation $P(t) = O + t\vec{D}$ into the sphere equation:

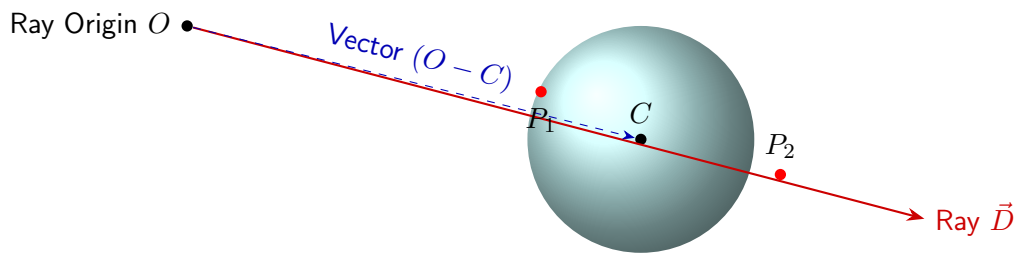
$$((O + t\vec{D}) - C) \cdot ((O + t\vec{D}) - C) = r^2$$

This equation expands into a quadratic equation in the form of $at^2 + bt + c = 0$, where:

- $a = \vec{D} \cdot \vec{D} = 1$ (since $\vec{D}$ is a unit vector)
- $b = 2\vec{D} \cdot (O - C)$
- $c = (O - C) \cdot (O - C) - r^2$

We can solve for t using the quadratic formula. The **discriminant** ($\Delta = b^2 - 4ac$) tells us the number of intersections:

- If $\Delta < 0$, the ray misses the sphere (no real solutions).
- If $\Delta = 0$, the ray grazes the sphere (one solution).
- If $\Delta > 0$, the ray passes through the sphere (two solutions). We are interested in the smallest, positive value of t , which represents the closest intersection point.



Ray Equation:

$$P(t) = O + t\vec{D}$$

Sphere Equation:

$$(P - C) \cdot (P - C) = r^2$$

Figure 1: A revised geometric representation for the ray-sphere intersection test, highlighting the key vectors and intersection points.

Ray-Sphere Intersection Test

```
1 // Ray-Sphere Intersection pseudo-code
2 function hitSphere(center, radius, ray) {
3   const oc = subtract(ray.origin, center);
4   const a = dot(ray.direction, ray.direction);
5   const b = 2.0 * dot(oc, ray.direction);
6   const c = dot(oc, oc) - radius*radius;
7   const discriminant = b*b - 4*a*c;
8
9   if (discriminant < 0) {
10    return -1.0;
11    // No hit
12  } else {
13    // Return t value of the closest hit point
14    return (-b - Math.sqrt(discriminant)) / (2.0 * a);
15  }
16 }
```

1.3 Simulating Light Phenomena

1.3.1 The Main Rendering Loop

A CPU-based ray tracer iterates through every pixel of the output image, generating a **primary ray** for each one that travels from the camera into the scene.

Main Ray Tracing Loop

```
1 // Pseudo-code for the main ray tracing loop
2 for (let j = 0; j < imageHeight; j++) {
3   for (let i = 0; i < imageWidth; i++) {
4     // 1. Calculate the ray for the current pixel (i, j)
5     const ray = camera.getRay(i, j);
6     // 2. Trace the ray to find the color
7     const color = trace(ray, world, depth);
8     // 3. Set the pixel color in the image buffer
9     setPixel(i, j, color);
10  }
11 }
```

1.3.2 Recursive Tracing for Reflections

To create mirror-like reflections, the ‘trace’ function can be made recursive. When a ray hits a reflective surface, we calculate the reflection direction and cast a new ray from the intersection point. The ‘trace’ function then calls itself with this new ray to find out what colour it sees. A depth limit is used to prevent infinite recursion between two mirrors, for instance.

Recursive Trace Function

```
1  // Recursive trace function pseudo-code
2  function trace(ray, world, depth) {
3      if (depth <= 0) return backgroundColor;
4      const hitRecord = world.intersect(ray);
5
6      if (hitRecord.hit) {
7          // ... calculate local color (ambient, diffuse) ...
8
9          if (hitRecord.material.isReflective) {
10             const reflectionRay = new Ray(hitRecord.point, hitRecord.reflectionDir);
11             const reflectionColor = trace(reflectionRay, world, depth - 1);
12             return localColor + reflectionColor * hitRecord.material.reflectivity;
13         }
14         return localColor;
15     }
16
17     return backgroundColor;
18 }
```

1.3.3 Casting Shadow Rays

Ray tracing produces sharp, accurate shadows naturally. After finding a ray-object intersection, we cast a special **shadow ray** from the intersection point towards each light source in the scene. If this shadow ray hits another object before reaching the light, the original point is in shadow and should not receive direct diffuse or specular illumination from that light. It may still be lit by ambient light or other light sources.

Shadow Ray Calculation

```
1  // Pseudo-code for shadow calculation inside the trace function
2  if (hitRecord.hit) {
3      const directionToLight = normalize(light.position - hitRecord.point);
4      const shadowRay = new Ray(hitRecord.point, directionToLight);
5  }
```

```
6  // Check for intersection up to the light's distance
7  if (world.intersect(shadowRay, distanceToLight)) {
8      // Something is blocking the light
9      return ambientColor;
10 } else {
11     // Not in shadow, calculate full lighting
12     return ambientColor + diffuseAndSpecularColor;
13 }
14 }
```

2 Section 2: Framebuffers & Post-Processing

2.1 Motivation: Applying Screen-Space Effects

So far, our rendering pipeline has always had one destination: the screen. The fragment shader calculates a colour, and that colour is written to the default framebuffer that is displayed on your monitor. This is limiting. What if we want to apply an effect to the entire rendered image at once, after all 3D geometry has been drawn? For example:

- Applying a blur effect to simulate motion or depth of field.
- Adjusting the brightness, contrast, or colour saturation of the entire scene.
- Creating effects like film grain, vignettes, or greyscale filters.

These operations, applied to the final 2D image after the 3D scene has been rendered, are known as **post-processing**. To achieve this, we need a way to redirect the output of the rendering pipeline, to render our scene to a texture instead of the screen.

2.2 The Framebuffer Object (FBO)

A **Framebuffer Object (FBO)** is a WebGL/OpenGL mechanism that allows us to render to an off-screen destination instead of the default screen buffer. An FBO is essentially a container for a collection of attachments. The most important attachment is a texture, which will act as our new colour buffer. When we bind a custom FBO, all subsequent drawing commands will write their fragment shader outputs into this texture. This enables a powerful two-pass rendering technique:

1. **Pass 1 (Scene Render):** We bind our custom FBO. We then render our entire complex 3D scene as usual. The final, rendered image is not sent to the screen, but is instead stored in the texture attached to our FBO.
2. **Pass 2 (Post-Processing):** We unbind the FBO, which directs rendering back to the screen. We then draw a simple, 2D rectangle that covers the entire screen (a "screen-filling quad"). We apply the texture we just rendered in Pass 1 to this quad. A new, simple fragment shader is used for this pass, which can perform any desired image manipulation by sampling from the scene texture.

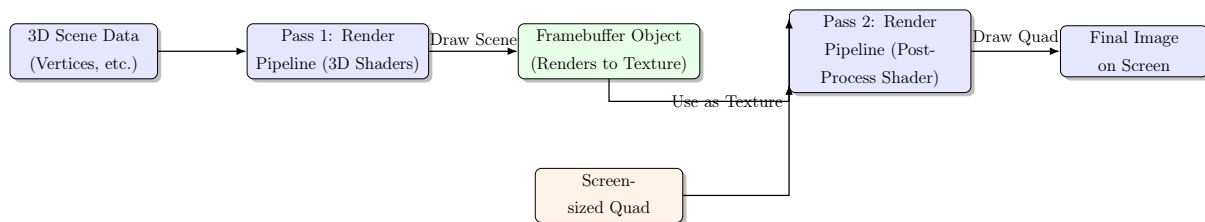


Figure 2: The two-pass rendering flow for post-processing. The scene is first rendered to a texture, which is then applied to a screen-sized rectangle and rendered to the screen with a final effect.

2.3 WebGL Implementation

2.3.1 Creating a Framebuffer

Setting up an FBO requires three steps: creating the texture that will serve as the colour buffer, creating the FBO itself, and attaching the texture to the FBO.

```

// ----- Framebuffer Object Creation -----
1  // 1. Create the texture to render to
2  const targetTexture = gl.createTexture();
3  gl.bindTexture(gl.TEXTURE_2D, targetTexture);
4  // Allocate storage for the texture, but don't fill it with data (null)
5  gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, width, height, 0, gl.RGBA, gl.UNSIGNED_BYTE,
   ↪  null);
6  // Set filtering parameters
7  gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR);
8
9  // 2. Create a framebuffer
10 const fb = gl.createFramebuffer();
11 gl.bindFramebuffer(gl.FRAMEBUFFER, fb);
12 // 3. Attach the texture to the framebuffer's color attachment point

```



```
13 gl.framebufferTexture2D(gl.FRAMEBUFFER, gl.COLOR_ATTACHMENT0, gl.TEXTURE_2D,  
    ↪ targetTexture, 0);
```

2.3.2 Switching Render Targets

You can switch between rendering to the FBO and rendering to the screen by binding the desired framebuffer. Binding 'null' reactivates the default screen buffer.

```
Switching Framebuffers  
1 // --- PASS 1: RENDER SCENE TO TEXTURE ---  
2 // Bind the framebuffer to draw to our texture  
3 gl.bindFramebuffer(gl.FRAMEBUFFER, fb);  
4 // ... Clear and draw your 3D scene as usual ...  
5 // The output is now in the 'targetTexture'  
6  
7 // --- PASS 2: RENDER TEXTURE TO SCREEN QUAD ---  
8 // Bind null to go back to drawing to the canvas  
9 gl.bindFramebuffer(gl.FRAMEBUFFER, null);  
10 // ... Set up a simple shader program for post-processing ...  
11 // ... Draw a screen-filling quad with the 'targetTexture' applied ...
```

2.4 Example Post-Processing Shaders

2.4.1 Greyscale Effect

A common method for converting a colour to greyscale is to average its red, green, and blue components. A more perceptually accurate method gives different weights to each component, reflecting human eye sensitivity: 'Luminance = $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$ '.

```
Greyscale Fragment Shader  
1 // Post-processing Fragment Shader  
2 precision mediump float;  
3 varying vec2 v_texCoord;  
4 uniform sampler2D u_sceneTexture;  
5 // The texture from Pass 1  
6  
7 void main() {
```

```
8   vec4 sceneColor = texture2D(u_sceneTexture, v_texCoord);
9   float average = (sceneColor.r + sceneColor.g + sceneColor.b) / 3.0;
10  gl_FragColor = vec4(average, average, average, 1.0);
11 }
```

2.4.2 Invert Colour Effect

Colour inversion is achieved by subtracting each colour component from its maximum value (1.0).

```
----- Invert Fragment Shader -----
1  // Post-processing Fragment Shader
2  precision mediump float;
3  varying vec2 v_texCoord;
4  uniform sampler2D u_sceneTexture;
5  void main() {
6      vec3 sceneColor = texture2D(u_sceneTexture, v_texCoord).rgb;
7      vec3 invertedColor = vec3(1.0) - sceneColor;
8      gl_FragColor = vec4(invertedColor, 1.0);
9  }
```

2.4.3 Box Blur Effect

A blur can be achieved using a *convolution kernel*. The idea is to sample the colours of neighbouring pixels and average them. A 3x3 box blur samples a grid of nine pixels (the current pixel and its eight immediate neighbours) and calculates their average colour. This is a simple but effective blur, though more advanced techniques like Gaussian blurs produce smoother results.

```
----- Box Blur Fragment Shader -----
1  // Post-processing Fragment Shader
2  uniform sampler2D u_sceneTexture;
3  uniform vec2 u_texelSize;
4  // The size of one pixel: (1.0/width, 1.0/height)
5  varying vec2 v_texCoord;
6
7  void main() {
8      vec4 total = vec4(0.0);
```

```
9  // Iterate over a 3x3 grid of pixels
10 for (int x = -1; x <= 1; x++) {
11     for (int y = -1; y <= 1; y++) {
12         // Calculate the UV coordinate of the neighboring pixel
13         vec2 offset = vec2(float(x), float(y)) * u_texelSize;
14         total += texture2D(u_sceneTexture, v_texCoord + offset);
15     }
16 }
17 // Average the 9 samples
18 gl_FragColor = total / 9.0;
19 }
```

3 Section 3: A Preview of Advanced Topics

Having mastered the core rendering pipeline, transformations, lighting, and texturing, you are now equipped to explore the more advanced techniques that are essential to modern computer graphics. This section provides a practical overview of several key topics, each of which builds directly on the fundamentals you have learned.

3.1 Shadow Mapping

While ray tracing produces perfect shadows, it is computationally expensive. **Shadow Mapping** is the standard rasterisation-based technique for generating shadows in real-time applications. Like post-processing, it is a multi-pass technique.

The Concept: The core idea is simple: if a point in the scene cannot be "seen" by the light source, it must be in shadow. We determine this by rendering the scene from the light's perspective first.

The Process:

1. **Pass 1 (Depth Pass):** The entire scene is rendered from the *point of view of the light source*. Instead of writing colours, this pass uses a special shader that writes each fragment's distance from the light into a texture. This texture is called the **shadow map**. This is another application of a Framebuffer Object.
2. **Pass 2 (Main Render):** The scene is rendered normally from the camera's point of view. In the fragment shader, for each fragment being rendered, we calculate its position from

the light's perspective. We use this position to look up the corresponding depth value in the shadow map (the distance of the closest object to the light in that direction). If the current fragment's distance from the light is greater than the value stored in the shadow map, it means there is another object closer to the light, and therefore the current fragment is in shadow.

Analysis: Shadow mapping is an ingenious use of Framebuffer Objects and coordinate space transformations. Its primary challenge lies in managing artefacts like "shadow acne" (self-shadowing errors) and aliasing (jagged shadow edges), which have led to many sophisticated variants of the basic algorithm (e.g., PCF, VSM).

3.1.1 Three.js Example: Shadow Mapping

Three.js abstracts the complex shadow mapping process, making it easy to implement. The key steps are enabling shadows on the renderer, marking which lights cast shadows, and specifying which objects can cast and receive shadows.

Three.js Shadow Mapping

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Three.js Shadow Mapping</title>
5   <style> body { margin: 0;
6     } canvas { display: block; } </style>
7 </head>
8 <body>
9   <script
10     ↪ src="https://cdn.jsdelivr.net/npm/three@0.132.2/build/three.min.js"></script>
11   <script>
12     // 1. Basic Scene Setup
13     const scene = new THREE.Scene();
14     const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
15       ↪ window.innerHeight, 0.1, 1000);
16     camera.position.set(5, 5, 5);
17     camera.lookAt(0, 0, 0);
18     const renderer = new THREE.WebGLRenderer({ antialias: true });
19     renderer.setSize(window.innerWidth, window.innerHeight);
20     // IMPORTANT: Enable shadow mapping on the renderer
21     renderer.shadowMap.enabled = true;
22     renderer.shadowMap.type = THREE.PCFSoftShadowMap; // Softer shadows
23     document.body.appendChild(renderer.domElement);
24     // 2. Add a light that casts shadows
```

```
23     const light = new THREE.DirectionalLight(0xffffff, 1);
24     light.position.set(5, 10, 7.5);
25     // IMPORTANT: Enable shadow casting for this light
26     light.castShadow = true;
27     scene.add(light);
28
29     // Optional: Add an ambient light for areas not in direct light
30     scene.add(new THREE.AmbientLight(0x404040));
31     // 3. Create objects that cast and receive shadows
32     // A ground plane to receive shadows
33     const planeGeo = new THREE.PlaneGeometry(20, 20);
34     const planeMat = new THREE.MeshStandardMaterial({ color: 0xcccccc });
35     const plane = new THREE.Mesh(planeGeo, planeMat);
36     plane.rotation.x = -Math.PI / 2;
37     plane.position.y = -1;
38     // IMPORTANT: Enable this object to receive shadows
39     plane.receiveShadow = true;
40     scene.add(plane);
41
42     // A sphere to cast a shadow
43     const sphereGeo = new THREE.SphereGeometry(1, 32, 32);
44     const sphereMat = new THREE.MeshStandardMaterial({ color: 0xaa3333 });
45     const sphere = new THREE.Mesh(sphereGeo, sphereMat);
46     sphere.position.y = 1;
47     // IMPORTANT: Enable this object to cast a shadow
48     sphere.castShadow = true;
49     scene.add(sphere);
50     // 4. Render Loop
51     function animate() {
52         requestAnimationFrame(animate);
53         sphere.rotation.y += 0.01;
54         renderer.render(scene, camera);
55     }
56     animate();
57 </script>
58 </body>
59 </html>
```

3.2 Skinned Animation

Skinned Animation (or skeletal animation) is the standard method for deforming and animating character meshes.

The Concept: A virtual "skeleton" made of hierarchical joints or **bones** is constructed inside the character mesh. Each vertex in the mesh is then "skinned" by assigning it influence weights to one or more nearby bones. For example, a vertex on a character's forearm might be 50% influenced by the elbow bone and 50% by the wrist bone.

The Process: To animate the character, an animator simply moves or rotates the bones of the skeleton. On the GPU, a vertex shader then calculates the final deformed position of each vertex by applying a weighted blend of the transformation matrices of its influencing bones. This allows for smooth, organic deformation of the mesh.

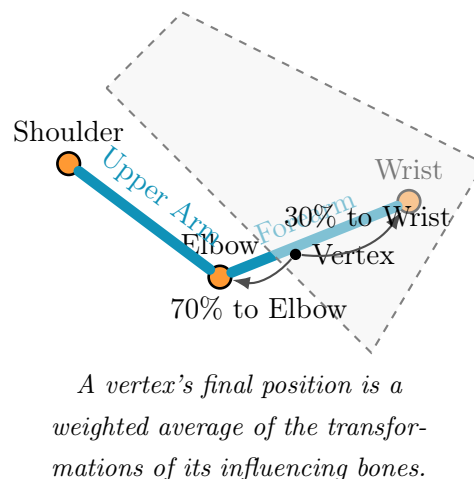


Figure 3: An illustration of Skinned Animation. A vertex on the mesh is influenced by multiple bones in the skeleton, with assigned weights determining its final position.

Analysis: This technique moves the complex, per-vertex deformation calculations from the CPU to the highly parallel GPU. The CPU only needs to update the transformation matrices for the relatively small number of bones in the skeleton each frame, making it extremely efficient for animating complex characters.

3.2.1 Three.js Example: Skinned Animation

Three.js handles skinned animation via its animation system and loaders like 'GLTFLoader'. When you load a 3D model that contains a skeleton and animations, Three.js automatically sets up the necessary components. Your main job is to play and update the animation.

Three.js Skinned Animation

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Three.js Skinned Animation</title>
5      <style> body { margin: 0;
6          } canvas { display: block; } </style>
7  </head>
8  <body>
9      <script type="importmap">
10         {
11             "imports": {
12                 "three":
13                     ↪ "https://cdn.jsdelivr.net/npm/three@0.132.2/build/three.module.js",
14                     "three/addons/":
15                     ↪ "https://cdn.jsdelivr.net/npm/three@0.132.2/examples/jsm/"
16             }
17         }
18     </script>
19     <script type="module">
20
21         import * as THREE from 'three';
22         import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
23         import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
24
25         // 1. Scene Setup
26         const scene = new THREE.Scene();
27         scene.background = new THREE.Color(0xdddddd);
28         const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
29             ↪ window.innerHeight, 0.1, 1000);
30         camera.position.set(0, 1.5, 3);
31         const renderer = new THREE.WebGLRenderer({ antialias: true });
32         renderer.setSize(window.innerWidth, window.innerHeight);
33         document.body.appendChild(renderer.domElement);
34
35         const controls = new OrbitControls(camera, renderer.domElement);
36
37         // 2. Lighting
38         const hemiLight = new THREE.HemisphereLight(0xffffff, 0x444444);
39         scene.add(hemiLight);
40         const dirLight = new THREE.DirectionalLight(0xffffff);
41         dirLight.position.set(5, 5, 5);
42         scene.add(dirLight);
43
44         // 3. Load Animated Model
45         let mixer;

```

```
42     // The animation mixer
43     const loader = new GLTFLoader();
44     // Model credit: "X Bot" from Mixamo.com
45
46     ↪ loader.load('https://raw.githubusercontent.com/mrdoob/three.js/dev/examples/models/gltf/
47     ↪ function (gltf) {
48         const model = gltf.scene;
49         scene.add(model);
50
51         // IMPORTANT: Create an AnimationMixer for this model
52         mixer = new THREE.AnimationMixer(model);
53
54         // Play the first animation clip found in the model
55         const action = mixer.clipAction(gltf.animations[0]);
56         action.play();
57
58     }, undefined, function (error) {
59         console.error(error);
60     });
61 // 4. Render Loop
62 const clock = new THREE.Clock();
63 function animate() {
64     requestAnimationFrame(animate);
65     // IMPORTANT: Update the animation mixer
66     const delta = clock.getDelta();
67     if (mixer) mixer.update(delta);
68
69     controls.update();
70     renderer.render(scene, camera);
71 }
72 animate();
73 </script>
</body>
</html>
```

3.3 Normal Mapping

Normal Mapping is a powerful texturing technique used to fake the appearance of high-resolution geometric detail on a low-polygon model.

The Concept: We use a special type of texture called a **normal map**. Instead of storing

colours, the RGB values of each texel in a normal map are interpreted as the X, Y, and Z components of a normal vector. This texture effectively stores a high-resolution normal for every point on the surface.

The Process: During the lighting calculations in the fragment shader, instead of using the (smoothly interpolated) normal from the low-polygon geometry, we sample the normal vector from the normal map. This high-resolution normal is then used for all lighting calculations (diffuse, specular, etc.). This tricks the lighting model into creating highlights and shadows as if the surface had much more intricate detail than it actually does.

Analysis: Normal mapping is a cornerstone of modern game art. It allows artists to create highly detailed models in sculpting programs and then "bake" that detail into a normal map that can be applied to a much simpler, game-ready model. This provides immense visual fidelity at a fraction of the performance cost of rendering the high-polygon model directly.

3.3.1 Three.js Example: Normal Mapping

Applying a normal map in Three.js is as simple as loading the texture and assigning it to the 'normalMap' property of a material like 'MeshStandardMaterial'. The material handles the complex fragment shader logic for you.

Three.js Normal Mapping

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Three.js Normal Mapping</title>
5      <style> body { margin: 0;
6          } canvas { display: block; } </style>
7  </head>
8  <body>
9      <script
10         ↪ src="https://cdn.jsdelivr.net/npm/three@0.132.2/build/three.min.js"></script>
11      <script>
12          // 1. Scene Setup
13          const scene = new THREE.Scene();
14          const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
15             ↪ window.innerHeight, 0.1, 1000);
16          camera.position.z = 5;
17          const renderer = new THREE.WebGLRenderer({ antialias: true });
18          renderer.setSize(window.innerWidth, window.innerHeight);
19          document.body.appendChild(renderer.domElement);
20          // 2. Lighting
21          const pointLight = new THREE.PointLight(0xffffff, 1);
```

```
20     pointLight.position.set(10, 10, 10);
21     scene.add(pointLight);
22     scene.add(new THREE.AmbientLight(0x222222));
23
24     // 3. Load Textures
25     const textureLoader = new THREE.TextureLoader();
26     const colorTexture =
27       ↳ textureLoader.load('https://threejs.org/examples/textures/brick_diffuse.jpg');
28     // IMPORTANT: Load the normal map texture
29     const normalTexture =
30       ↳ textureLoader.load('https://threejs.org/examples/textures/brick_bump.jpg');
31     colorTexture.wrapS = THREE.RepeatWrapping;
32     colorTexture.wrapT = THREE.RepeatWrapping;
33     normalTexture.wrapS = THREE.RepeatWrapping;
34     normalTexture.wrapT = THREE.RepeatWrapping;
35     // 4. Create Material and Object
36     const geometry = new THREE.BoxGeometry(2, 2, 2);
37     // Use a material that supports normal maps
38     const material = new THREE.MeshStandardMaterial({
39       map: colorTexture,
40       // IMPORTANT: Assign the normal map
41       normalMap: normalTexture
42     });
43     const cube = new THREE.Mesh(geometry, material);
44     scene.add(cube);
45
46     // 5. Render Loop
47     function animate() {
48       requestAnimationFrame(animate);
49       cube.rotation.x += 0.005;
50       cube.rotation.y += 0.005;
51       // Move light to show effect
52       const time = Date.now() * 0.001;
53       pointLight.position.x = Math.sin(time) * 10;
54       pointLight.position.z = Math.cos(time) * 10;
55       renderer.render(scene, camera);
56     }
57     animate();
58   </script>
59 </body>
60 </html>
```

3.4 Physically Based Rendering (PBR)

Physically Based Rendering (PBR) is a modern approach to shading that aims to model the interaction of light and materials in a more physically accurate manner than traditional models like Blinn-Phong.

The Concept: Rather than using ad-hoc artistic parameters like "shininess", PBR materials are defined by physically meaningful properties. The two main workflows are *Metallic/Roughness* and *Specular/Glossiness*. In the metallic workflow, the core properties are:

- **Albedo:** The base colour of the surface.
- **Metallic:** A value from 0 (dielectric) to 1 (metal).
- **Roughness:** A value from 0 (smooth, sharp reflections) to 1 (rough, blurry reflections).

The Process: A PBR shader takes these properties (often from textures) and feeds them into a complex lighting equation based on microfacet theory, such as the Cook-Torrance BRDF. This model also enforces energy conservation, ensuring a surface cannot reflect more light than it receives, which prevents overly bright highlights.

Analysis: PBR has become the industry standard because it allows artists to create materials that look correct and realistic under any lighting condition, leading to more consistent and believable visuals. It often relies on image-based lighting (environment maps) to provide realistic reflections.

3.4.1 Three.js Example: Physically Based Rendering

Three.js implements PBR through the 'MeshStandardMaterial' and 'MeshPhysicalMaterial'. The key is to provide physically-based parameters and, for best results, an environment map for reflections.

Three.js PBR

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Three.js PBR</title>
5   <style> body { margin: 0;
6     } canvas { display: block; } </style>
7 </head>
8 <body>
9   <script type="importmap">
```

```

10     {
11         "imports": {
12             "three":
13                 ↪ "https://cdn.jsdelivr.net/npm/three@0.132.2/build/three.module.js",
14             "three/addons/":
15                 ↪ "https://cdn.jsdelivr.net/npm/three@0.132.2/examples/jsm/"
16         }
17     }
18 }
19 </script>
20 <script type="module">
21
22     import * as THREE from 'three';
23     import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
24     import { RGBELoader } from 'three/addons/loaders/RGBELoader.js';
25
26     // 1. Scene Setup
27     const scene = new THREE.Scene();
28     const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
29         ↪ window.innerHeight, 0.1, 1000);
30     camera.position.z = 10;
31     const renderer = new THREE.WebGLRenderer({ antialias: true });
32     renderer.setSize(window.innerWidth, window.innerHeight);
33     renderer.toneMapping = THREE.ACESFilmicToneMapping;
34     // More realistic lighting
35     renderer.toneMappingExposure = 1;
36     document.body.appendChild(renderer.domElement);
37     const controls = new OrbitControls(camera, renderer.domElement);
38
39     // 2. PBR Environment Map
40     // An environment map provides realistic ambient light and reflections.
41     new RGBELoader()
42         .setPath('https://threejs.org/examples/textures/equirectangular/')
43         .load('royal_esplanade_1k.hdr', function (texture) {
44             texture.mapping = THREE.EquirectangularReflectionMapping;
45             scene.background = texture;
46             scene.environment = texture;
47         });
48     // 3. PBR Material and Objects
49     const sphereGeo = new THREE.SphereGeometry(1, 64, 64);
50     // Create a grid of spheres with varying metallic and roughness
51     for (let i = 0; i <= 5; i++) {
52         for (let j = 0; j <= 5; j++) {
53             const material = new THREE.MeshStandardMaterial({
54                 color: 0xffaa00,

```

```
51
52      // IMPORTANT: Vary roughness and metalness
53      roughness: i / 5.0,
54      metalness: j / 5.0
55    });
56    const sphere = new THREE.Mesh(sphereGeo, material);
57    sphere.position.x = (i - 2.5) * 2.5;
58    sphere.position.y = (j - 2.5) * 2.5;
59    scene.add(sphere);
60  }
61 }
62
63 // 4. Render Loop
64 function animate() {
65   requestAnimationFrame(animate);
66   controls.update();
67   renderer.render(scene, camera);
68 }
69 animate();
70 </script>
71 </body>
72 </html>
```

3.5 Particle Systems

Particle Systems are used for simulating "fuzzy" or chaotic phenomena like fire, smoke, sparks, rain, and explosions.

The Concept: A particle system manages a large number of very simple objects, called **particles**. Each particle is an independent entity with attributes like position, velocity, colour, size, and lifetime.

The Process: An *emitter* generates new particles. In each frame, the state of every active particle is updated according to rules and physical forces (e.g., gravity, wind). This update can happen on the CPU or GPU. After the update, particles are rendered, typically as small, camera-facing 2D sprites (billboards).

Analysis: Particle systems are a powerful example of procedural animation. By defining simple rules for individual particles, complex and organic-looking emergent behaviour can be created for the system as a whole. GPU-based systems can handle millions of particles for large-scale

effects.

3.5.1 Three.js Example: Particle System

Three.js can create particle systems using the ‘Points’ object and a ‘PointsMaterial’. The logic for updating particle positions is typically handled on the CPU in the animation loop.

```
Three.js Particle System
```

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Three.js Particle System</title>
5      <style> body { margin: 0;
6          background-color: #000; } canvas { display: block; } </style>
7  </head>
8  <body>
9      <script
10         ↪ src="https://cdn.jsdelivr.net/npm/three@0.132.2/build/three.min.js"></script>
11      <script>
12          // 1. Scene Setup
13          const scene = new THREE.Scene();
14          const camera = new THREE.PerspectiveCamera(75, window.innerWidth /
15             ↪ window.innerHeight, 0.1, 1000);
16          camera.position.z = 50;
17
18          const renderer = new THREE.WebGLRenderer();
19          renderer.setSize(window.innerWidth, window.innerHeight);
20          document.body.appendChild(renderer.domElement);
21
22          // 2. Create Particles
23          const particleCount = 5000;
24          const positions = new Float32Array(particleCount * 3);
25          const velocities = [];
26          // Use a standard array for JS objects
27
28          const particleGeometry = new THREE.BufferGeometry();
29          for (let i = 0; i < particleCount; i++) {
30              // Set initial position
31              positions[i * 3 + 0] = (Math.random() - 0.5) * 100;
32              // x
33              positions[i * 3 + 1] = (Math.random() - 0.5) * 100;
34              // y
35              positions[i * 3 + 2] = (Math.random() - 0.5) * 100;
36              // z
```

```
35
36      // Store initial velocity for CPU update
37      velocities.push({
38          x: (Math.random() - 0.5) * 0.1,
39          y: (Math.random() - 0.5) * 0.1,
40
41          z: (Math.random() - 0.5) * 0.1,
42      });
43  }
44
45  particleGeometry.setAttribute('position', new
    ↪ THREE.BufferAttribute(positions, 3));
46  // 3. Create Material
47  const particleMaterial = new THREE.PointsMaterial({
48      color: 0xffffff,
49      size: 0.5,
50      transparent: true,
51      blending: THREE.AdditiveBlending // Good for sparks/stars
52  });
53  // 4. Create the Points object
54  const particles = new THREE.Points(particleGeometry, particleMaterial);
55  scene.add(particles);
56  // 5. Render Loop with CPU update
57  function animate() {
58      requestAnimationFrame(animate);
59      // IMPORTANT: Update particle positions
60      const positionsAttribute = particles.geometry.attributes.position;
61      for (let i = 0; i < particleCount; i++) {
62          // Apply velocity
63          positionsAttribute.array[i * 3 + 0] += velocities[i].x;
64          positionsAttribute.array[i * 3 + 1] += velocities[i].y;
65          positionsAttribute.array[i * 3 + 2] += velocities[i].z;
66          // Simple boundary check to reset particles
67          if (positionsAttribute.array[i * 3 + 1] < -50) {
68              positionsAttribute.array[i * 3 + 1] = 50;
69          }
70      }
71      positionsAttribute.needsUpdate = true;
72      // Tell three.js to update the GPU buffer
73
74      particles.rotation.y += 0.001;
75      renderer.render(scene, camera);
76  }
77  animate();
```

```
78     </script>
79 </body>
80 </html>
```

4 Section 4: Reflective Questions: The Future of Graphics

As we conclude this course, you have transitioned from drawing simple triangles to understanding the mechanics of modern, photorealistic engines. To guide your future learning, consider these challenges that define the current cutting edge of Computer Graphics.

- **The Hybrid Horizon (Ray Tracing + Rasterisation):** We contrasted Rasterisation (fast, approximate) with Ray Tracing (slow, accurate). Modern GPUs (like the NVIDIA RTX series) can now do some ray tracing in real-time, but not enough for a whole scene. *How do modern engines combine these two philosophies? Could we use Rasterisation for the main geometry, but fire rays only for specific effects like reflections and shadows? (Search term: Hybrid Rendering).*
- **The Neural Revolution (AI in Graphics):** We learned about standard rendering pipelines. However, AI is changing everything. *Instead of rendering pixels explicitly, what if we trained a Neural Network to "guess" what a high-resolution image looks like based on a low-resolution input? Or what if we replaced 3D meshes entirely with a neural representation of a volume? (Search terms: DLSS, NeRFs, Gaussian Splatting).*
- **The Efficiency Paradox (Deferred Shading):** In our loop, we shade every fragment generated. But what if we render a complex object, and then it immediately gets covered up by a wall drawn later? We wasted time calculating lighting for the hidden object. *How can we use Framebuffers to save all the raw data (positions, normals, colours) to textures first, and then perform lighting calculations only once per pixel on the final screen image? (Search term: Deferred Shading).*
- **The Content Crisis (Proceduralism vs. Hand-Crafted):** We looked at Particle Systems and PBR. As 4K and 8K worlds demand more detail, it becomes impossible for artists to place every rock and paint every texture by hand. *How can we write algorithms that generate entire cities, forests, or planets automatically using the rules of nature and mathematics? (Search term: Procedural Generation / Houdini).*

"The definition of the possible is the definition of the artist's future."

— Sutherland's Sketchpad (1963), The ancestor of all modern GUIs and Graphics.

End of Module

Congratulations on completing COMP2271 Computer Graphics.